Лабораторная работа 1

Запуск проекта · Каменский Андрей Игоревич · БПИ2301 · 10.11.2026

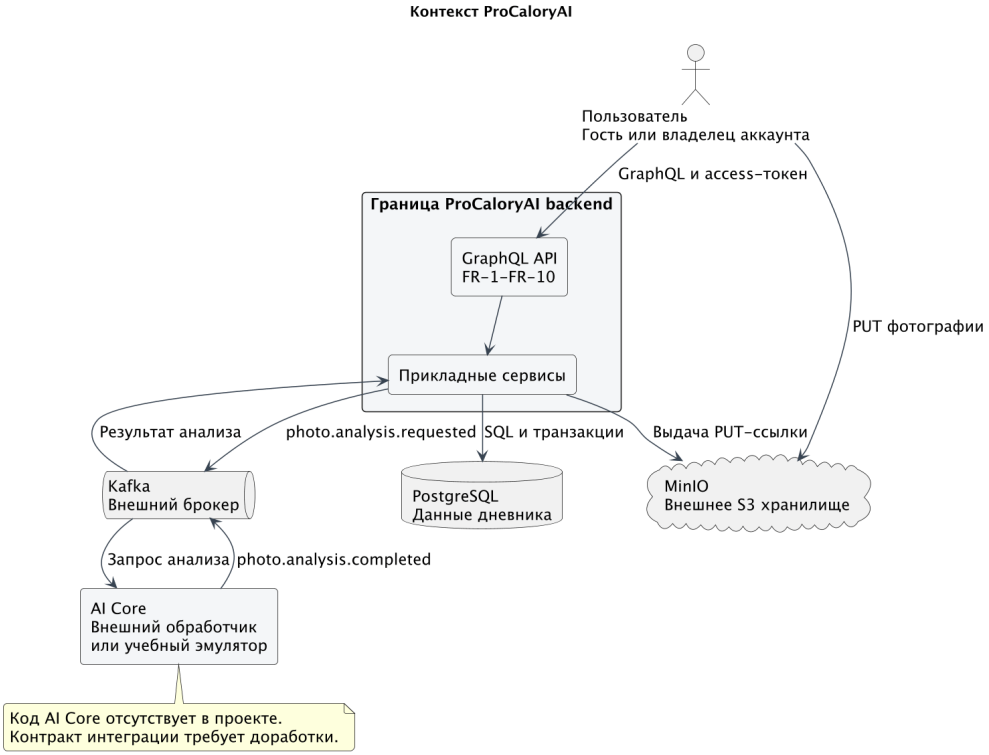
Проект и основные функции

ProCaloryAI — мой сервер дневника питания. Пользователь записывает еду, смотрит КБЖУ и задаёт цель снижения веса. Проект собственный; работаю индивидуально. Репозиторий: github.com/gonnem2/ProCaloryAI-backend.

Три основные функции: ручной дневник, статистика питания, цель. Одна роль — владелец аккаунта; до входа пользователь — гость. Для расширения предусмотрен внешний AI Core через Kafka и MinIO. Сейчас Core отсутствует, фотоанализ отложен.

Пять сущностей: User, MealLog, Goal, AnalysisRequest, Preferences. Настройки приватности — вспомогательные данные. **Стек:** Python 3.13.9, uv 0.9.5, FastAPI, GraphQL, PostgreSQL 17.6, SQLAlchemy, Alembic. LICENSE сохраняет авторские права.

Граница системы



Проект подходит под курс: собственная предметная область, одна роль и внешние системы, пять основных сущностей и десять пользовательских требований. Внешние сервисы фотоанализа не нужны для работы ручного дневника.

Запуск и результат проверки

Нужны Docker Engine и Compose v2. С получением проекта — пять команд. Из текущей папки достаточно запуска Compose и проверки health. Исправления пока локальные; для проверки из клона их нужно перенести в GitHub.

```
git clone https://github.com/gonnem2/ProCaloryAI-backend.git
cd ProCaloryAI-backend
cp .env.example .env
docker compose up -d --build --wait
curl --fail http://localhost:8000/health
```

Сценарий: 1) register; 2) с access-токеном добавить кашу 250/8/5/42 через addMealLog; 3) запросить meals и todayStats. В чистом дневнике видна одна запись и те же суммы. Запросы есть в scripts/smoke.graphql; IDE: localhost:8000/graphql.

Проверено 10 ноября 2026 года

macOS, Linux-контейнеры с новой БД. Docker 28.5.1, Compose 2.40.0-desktop.1. Образ собран, миграция выполнена, /health отвечает ok. Прошли 11 API-тестов. Запись сохранилась после перезапуска backend. Kafka, MinIO и Core не запускались.

```
test_01_health_and_schema (test_api.DiaryAPI.test_01_health_and_schema) ... ok
test_02_login_and_refresh (test_api.DiaryAPI.test_02_login_and_refresh) ... ok
test_03_bad_credentials_and_token_type (test_api.DiaryAPI.test_03_bad_credentials_and_token_type) ... ok
test_04_duplicate_registration (test_api.DiaryAPI.test_04_duplicate_registration) ... ok
test_05_manual_diary_stats_and_ownership (test_api.DiaryAPI.test_05_manual_diary_stats_and_ownership) ... ok
test_06_validation_without_write (test_api.DiaryAPI.test_06_validation_without_write) ... ok
test_07_week_boundary_and_timezone (test_api.DiaryAPI.test_07_week_boundary_and_timezone) ... ok
test_08_goal_create_update_validation_and_progress (test_api.DiaryAPI.test_08_goal_create_update_validation_and_progress) ... ok
test_09_profile_preferences_and_privacy (test_api.DiaryAPI.test_09_profile_preferences_and_privacy) ... ok
test_10_photo_disabled_and_pagination (test_api.DiaryAPI.test_10_photo_disabled_and_pagination) ... ok
test_11_registration_validation (test_api.DiaryAPI.test_11_registration_validation) ... ok

----------------------------------------------------------------------
Ran 11 tests in 0.976s

OK
```

Рисунок — фактический вывод unittest в контейнере. Повторить: docker compose exec -T backend python -m unittest discover -s tests -v.

Устройство и карта функций

Функции	Файл в src/application/	Проверка
Регистрация, вход, refresh	service/user.py	register → login → refreshTokens
Запись, дневник, удаление	service/meal_log.py	addMealLog → meals → deleteMealLog
Статистика дня и недели	service/meal_log.py	todayStats; weeklyAverage
Цель и прогресс	service/goal.py	upsertGoal → goal
Настройки и профиль	service/profile.py	updatePreferences; preferences; me; achievements
Фотоанализ по плану	service/analysis.py	Пока сообщение о недоступности Core

Точка входа: src/main.py. API: src/api/graphql/. Сущности: src/domain/models/. База, репозитории и транзакции: src/infrastructure/. Миграции: src/migrations/. Проверки: tests/.

Проблемы и план работы

Инженерные практики

Практика	Состояние
Тесты	11 API-тестов unittest проходят
Линтер	Ruff: критические ошибки проверены
Зависимости и миграции	uv.lock и Alembic; версии инструментов закреплены
Документация	README, docs, CHANGELOG и короткие PDF
CI	Пока нет
GitHub	Настройки main, issues, PR и теги ещё нужно оформить

База данных хранится в именованном Docker-томе; параметры передаются через окружение. Старые папки postgres_data и minio_data не менялись. Пароли — bcrypt-хеши, ключ JWT берётся из окружения.

Первые найденные проблемы

1. Мутации обращались к user.id вместо поля словаря — исправлено.
2. Недельная статистика содержала неверный SQL и делитель — исправлено.
3. Backend ждал Kafka даже без Core — исправлено, добавлен режим без фото.
Открыто: интеграция Core, защита от повторных результатов анализа, CI. Тексты issues подготовлены в docs/issues.md.

Выполнимость

Риск	Оценка и решение
Core не появится	Высокая вероятность; ручной дневник — основной объём, фото отложено
Не хватит времени	Средняя; сначала FR-1–FR-7, затем дополнительные функции
Ошибки интеграций	Средняя; будущий контракт Core и при необходимости эмулятор

План: работа 2 — требования и модели, 8 ч; работа 3 — уточнение и архитектура, 10 ч; работа 4 — доработки, 12 ч; работа 5 — тестирование, 10 ч; работа 6 — CI и выпуск, 8 ч. Все задачи выполняю сам, резерв 10 ч. Работы 3–6 уточняются по их методичкам.

Осталось по процедуре: согласовать product с преподавателем, разместить минимум три issue, получить внешнюю проверку запуска, провести PR lab-1 и поставить тег lab-1. Эти действия ещё не выполнены. Код и локальная проверка готовы.